The ENIGMA Team

Google Summer of Code - 2022
Project Proposal

Data Buffers / Serialization

About me

Name: Dhruv Chawla
Discord ID: Dhruv Chawla#7049
Email: -
University: -
Major: -
GitHub: -
Time Zone: -

Abstract

The ENIGMA project currently does not implement a simple method to save the game state into a file. While this can be done using the pre-existing Binary Buffer system, it is a bulky solution which requires a lot of boilerplate code to set up the buffers, write the object data into them, and then write the buffer to a file.

This project can be divided into two parts:

- Part i. Finish up and polish the Binary Buffer system's API
    - Fix / add serialization for floating point and complex types
    - Add MD5 and SHA1 encryption functions, and implement the base64 functions
    - Create tests to ensure alignment, serialization and deserialization work properly
- Part ii. Begin implementing the game save functions
    - Emit functions to serialize and deserialize user variables
    - Emit functions to serialize internal engine data structures
    - Create the `game_save_buffer` and `game_load_buffer` functions to save and load the entire game state

The changes to the Binary Buffer system will be applied under the https://github.com/enigma-dev/enigma-dev/tree/master/ENIGMAsystem/SHELL/Universal_System directory.

The changes to the compiler's code generator will be applied under the https://github.com/enigma-dev/enigma-dev/tree/master/CompilerSource/compiler/components directory.

Implementation Plan

`Part i. The Binary Buffer API`

1. Buffer layout

- The `BinaryBuffer` will be updated to store metadata along with the bytes themselves, so that the types being stored into the buffer can be known.

2. Floating point and complex numbers

- The current implementation of the `valToBytes` function used by the `buffer_poke` and `buffer_write` contains a bug where floating point numbers have their decimal digits truncated, causing them to be treated as normal integers. This is caused by the implementation of `operator>>` in the `variant` class located at https://github.com/enigma-dev/enigma-dev/blob/master/ENIGMAsystem/SHELL/Universal_System/var4.h#L568-L571

- Thus, I would begin by fixing this bug. This would involve either doing a byte-by-byte copy using an equivalent of `std::bit_cast` or by defining an encoder and decoder for the IEEE-754 format from the platform-dependent floating point type.

- A new buffer constant will be added, `buffer_comp` (representing complex numbers), and a special case will be added to the `valToBytes` function to serialize complex numbers as two integers or two floating point numbers.

3. MD5, SHA1 and base64 functions

- The following functions are currently unimplemented:
  - `buffer_base64_decode`

    - `buffer_base64_decode_ext`
    - `buffer_base64_encode`
    - `buffer_md5`
    - `buffer_sha1`

- These functions will be implemented in the order listed.

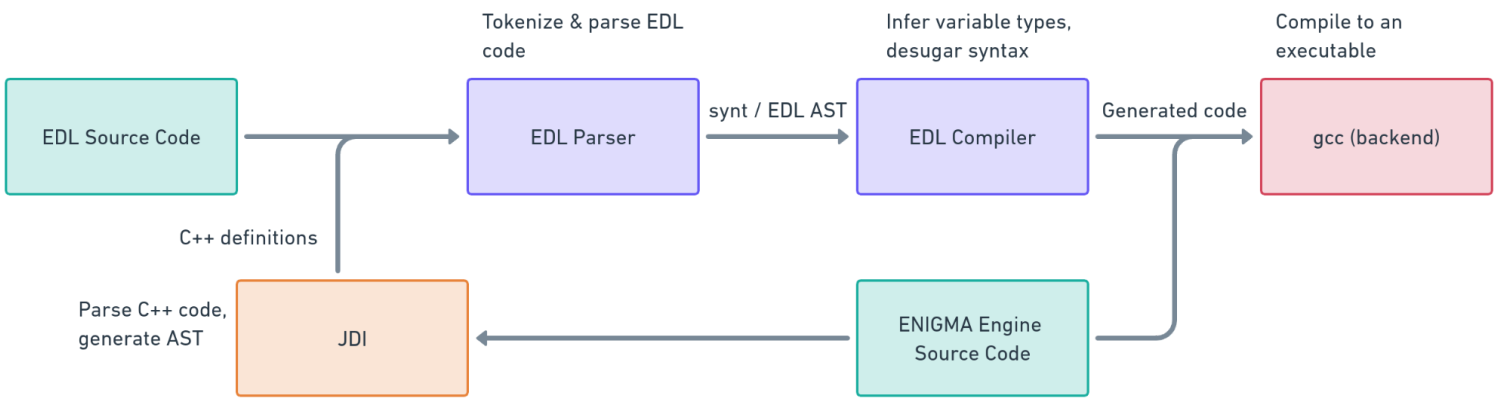
4. Testing

- Tests will be created in a new `tests/` directory under the main `Universal_System` directory

---

Part ii. The EDL Compiler

The compilation of EDL currently works in the following way:



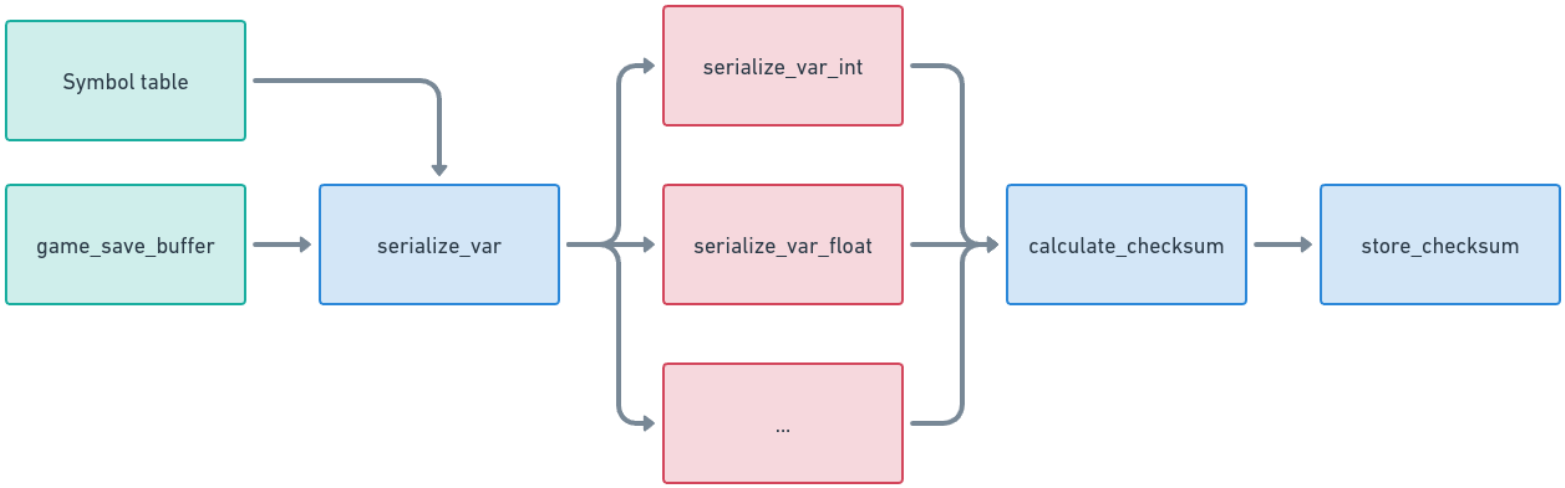
The second part of the project will focus on the EDL compiler, by adding the ability to serialize and deserialize all game state to a buffer. This will require a special buffer format, where a header containing a symbol table will be generated first, defining how data in the buffer maps to variables in the source.

This symbol table will be represented in code as an `unordered_map<string, pair<string, void*>>` or its equivalent, representing a mapping of variable names to a pair of the variable's type and a pointer to the actual variable.

Serialization: For serialization, the compiler will emit type-specific functions to convert a given variable, and then iterate over the symbol table to store each variable into the provided buffer.

A checksum will then be calculated from the data stored in the buffer, and will be written to the header for checking data validity.

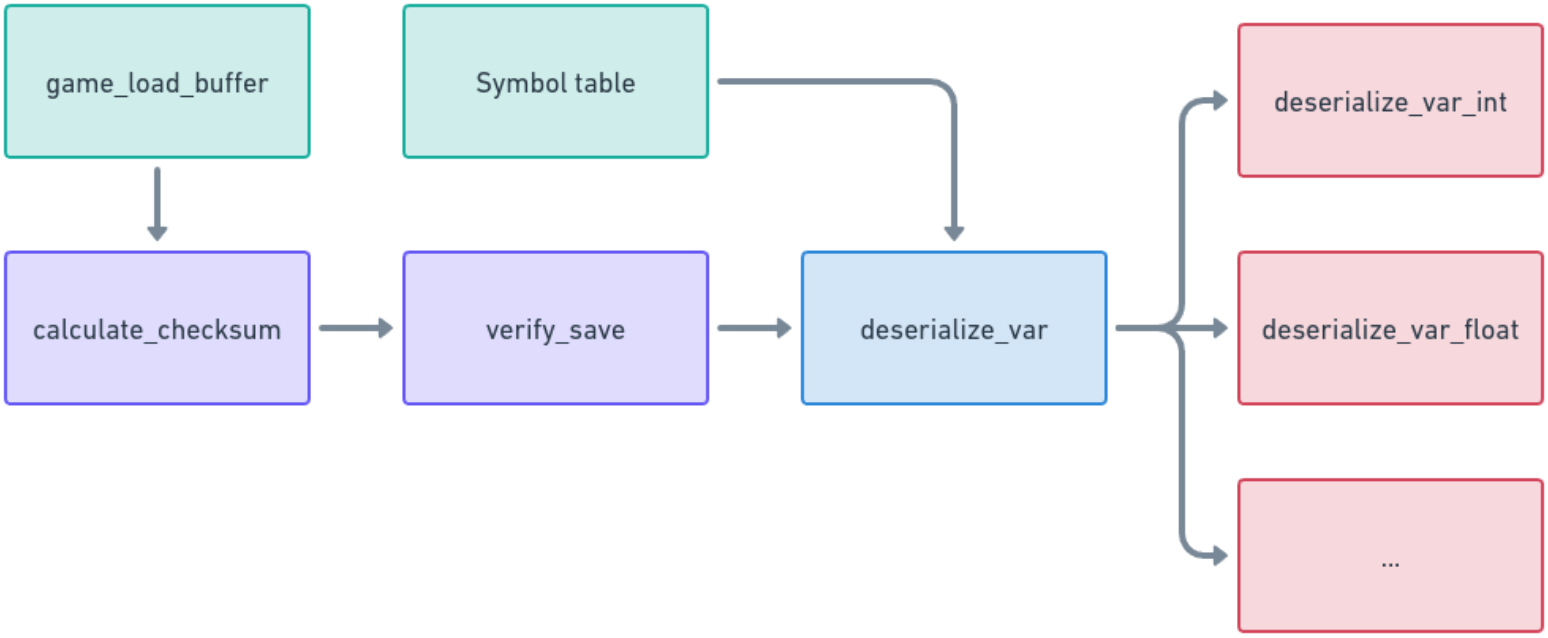
Therefore, the flow of control will look like this:



Deserialization: For deserialization, the compiler will emit a C++ function to iterate over the symbol table stored in the buffer and use it to access the symbol table in code so that variable addresses and their types can be accessed and written to.

An additional function, `verify_save`, will be used to ensure that the game state header is valid. This function will verify the checksum stored in the header.

Therefore, the flow of control will look like this:



At the end of each week, I will create a post on my github pages website outlining all the work or research I have done throughout the previous week.

Proposal Timeline

- **Before April 20:**
  - Practice creating basic games such as pong using EDL
  - Start exploring the code base to understand how the game engine works internally

- **May 20 - June 12 (Before the official coding time):**
  - Gain a better understanding of EDL by discussing details of the syntax and code generation with the mentors and exploring the `JDI` parser.
  - Create an architecture diagram of ENIGMA as a whole and add it to the wiki for easier exploration.
  - Finalize all the required modifications to the Binary Buffer system as well as the compiler.
  - Decide on a simple and portable format for the serialization of floating point and complex number types.

- **June 13 - July 1 (Phase 1 standard coding period):**
  - Implement all the required Binary Buffer system functions and refactor the implementations of the current functions wherever required. Submit a PR for the created functions.
  - Write tests for the functions. Create a PR for them.
  - Modify the wiki with better, more comprehensive documentation for all the functions.

- **July 1 - July 25 (Phase 1 standard coding period):**
  - Review the changes to the Binary Buffer API, and refactor with better code where necessary. Submit a PR for any refactors done.
  - Start work on the EDL compiler, by implementing the symbol table along with export and import functions. This includes the checksum calculation and verification functions.

- **July 25 - August 25 (Work Period):**
  - Implement the code generation for the type-specific serialization and deserialization functions and implement the game save and load functions. Submit a PR for the changes.
  - Write tests to ensure game data before serialization and after deserializing from a file are equivalent. Submit a PR for the changes.
  - Create documentation for how the game state is serialized and what data format is used on the wiki with the help of a data layout diagram.

- **August 25 - September 12 (Work Period):**
  - Review the changes to the compiler's code, and refactor with better code wherever necessary.
  - Submit a final PR containing any refactors done.

- **September 12 - September 19 (Standard coding period):**
  - Buffer period, in case any emergencies occur.